

# Documentação da Pipeline CI/CD para o Projeto Jakebot

---

## Introdução

---

Este documento detalha a pipeline de Integração Contínua (CI) e Entrega Contínua (CD) proposta para o projeto Jakebot. A pipeline foi projetada para automatizar o processo de construção, teste e implantação do projeto, garantindo a qualidade do código e a agilidade nas entregas.

## Tecnologias Utilizadas

---

O projeto Jakebot é composto por duas partes principais: - **Frontend (Jekyll):** Um blog estático construído com Jekyll, que utiliza Ruby. - **Backend (Django):** Uma aplicação web desenvolvida com Django, que utiliza Python.

Com base nessas tecnologias, a pipeline foi configurada para utilizar o GitHub Actions, uma ferramenta de CI/CD integrada ao GitHub, que permite automatizar fluxos de trabalho diretamente no repositório.

## Estrutura da Pipeline

---

A pipeline é definida no arquivo `.github/workflows/main.yml` e consiste em três jobs principais:

### 1. `build_jekyll`

Este job é responsável por construir e implantar o site estático do Jekyll. Ele executa as seguintes etapas: - **Checkout do Código:** Clona o repositório do projeto. - **Configuração do Ruby:** Configura o ambiente Ruby e instala as dependências do Bundler. - **Construção do Site Jekyll:** Executa os comandos `bundle install` e

`bundle exec jekyll build` para construir o site. - **Deploy Jekyll para GitHub Pages:** Utiliza a action `peaceiris/actions-gh-pages@v3` para publicar o site construído no GitHub Pages. É importante que o `baseurl` no `_config.yml` esteja configurado corretamente para o subdiretório do repositório (ex: `/2025_1-Jakebot`).

## 2. `build_and_test_django`

Este job é responsável por configurar o ambiente Python, instalar as dependências do Django e executar os testes da aplicação backend. Ele executa as seguintes etapas: - **Checkout do Código:** Clona o repositório do projeto. - **Configuração do Python:** Configura o ambiente Python. - **Instalação de Dependências:** Instala as dependências listadas no arquivo `requirements.txt`. - **Execução dos Testes Django:** Navega até o diretório `backend` e executa os testes da aplicação Django com `python manage.py test`.

## 3. `deploy_django`

Este job é responsável pela implantação da aplicação Django. Ele depende do sucesso do job `build_and_test_django`. As etapas são: - **Checkout do Código:** Clona o repositório do projeto. - **Configuração do Python:** Configura o ambiente Python. - **Instalação de Dependências:** Instala as dependências listadas no arquivo `requirements.txt`. - **Deploy Django (Placeholder):** Esta etapa contém um placeholder para os comandos de implantação do Django. Você precisará adicionar os comandos específicos para o seu ambiente de deploy (ex: Heroku, AWS Elastic Beanstalk, Google App Engine). Exemplos de comandos para Heroku são fornecidos como comentário no arquivo da pipeline.

## Gatilho da Pipeline

---

A pipeline é acionada automaticamente em cada `push` para a branch `main` do repositório. Isso garante que qualquer alteração no código principal seja automaticamente testada, validada e, agora, implantada.

## Como Utilizar

---

Para utilizar esta pipeline, siga os passos: 1. Certifique-se de que o arquivo `.github/workflows/main.yml` esteja na raiz do seu repositório. 2. Configure o GitHub Pages para o seu repositório, se ainda não o fez. 3. Para o deploy do Django, adicione os comandos de implantação específicos para o seu provedor de hospedagem na seção `Deploy Django (Placeholder)` do job `deploy_django`. 4. Faça um `push` para a branch `main`. 5. Acompanhe o status da execução da pipeline na aba "Actions" do seu repositório GitHub.